JSQKV: Joint Sparsification and Quantization for KV-Cache Compression and Decode Acceleration

Anonymous Authors

Draft prepared for APPT 2026 submission

Abstract. As long-context large language models become increasingly widespread, autoregressive decoding is increasingly bottlenecked by KV-cache footprint and memory bandwidth. Existing KV-cache compression methods typically rely on sparsification or low-bit quantization, but simply combining the two often fails to simultaneously preserve model accuracy and deliver real system-level gains. We present JSQKV, a joint sparse-quant framework for KV-cache compression and decode acceleration that co-designs compression policy, numerical representation, and execution path. JSQKV introduces an attention-score-based differential sparsification strategy with a dual-window online execution mechanism, together with a Hadamard-based Per-Token-Tile quantization scheme and a bitmap-based sparse-quant format with a matching decode kernel. This co-design enables compressed KV-cache data to remain compact in memory while being effectively translated into reduced memory traffic and end-to-end speedup. Experimental results show that, under the same compression budget, JSQKV consistently outperforms the sequential MUSTAFAR+KIVI baseline on LongBench. On Meta-Llama-3-8B, with input length 4096 and output length 256, JSQKV improves end-to-end throughput by 44.2% over the dense baseline at batch size 4 using 70% KV sparsity and 2-bit quantization, while increasing the KV-cache compression ratio to 3.48×. These results show that efficient KV-cache compression requires joint optimization across compression policy, numerical representation, and execution path. Code is available at https://github.com/Haozon/JSQKV.

Keywords: KV-cache compression · Sparsity · Quantization

1 Introduction

As context windows continue to grow in large language models, autoregressive decoding is increasingly bottlenecked by KV-cache footprint and memory bandwidth. The KV cache grows linearly with context length, while decoding offers limited parallelism and repeatedly accesses the KV cache, making it more memory-bound than the prefill stage. In long-context settings, this bottleneck further limits throughput, batch size, and deployment efficiency.

Prior work on KV-cache optimization mainly follows two directions. One line reduces the amount of retained history through token eviction or sparsification.

Representative methods such as StreamingLLM [12], H_2O [14], SnapKV [9], and PyramidKV [3] retain only selected historical tokens, while ThinK [13], KVPruner [11], and MUSTAFAR [8] further explore channel-wise, structured, and unstructured sparsity. These studies show that historical tokens are not equally important and that importance-aware cache reduction can effectively relieve memory and bandwidth pressure. The other line compresses the numerical representation of KV-cache elements through low-bit quantization. KVQuant [7], QuaRot [1], and KIVI [10] explore ultra-low-bit KV representation, distribution reshaping, and asymmetric quantization for Keys and Values. These methods directly reduce KV-cache footprint and bandwidth cost, but they mainly focus on how to quantize each cache element rather than which historical information should be retained.

Overall, prior work shows that both sparsification and quantization are effective, but the two are still largely studied in isolation. In practice, joint compression is not a simple sequential combination: once sparsification changes the retained tokens and cache access pattern, the appropriate quantization granularity, compressed data layout, and decode-time kernel design also change. Without such co-design, algorithm-level compression gains may not reliably translate into practical end-to-end speedups.

To address this issue, we propose JSQKV, a joint sparse-quant framework for KV-cache compression and decode acceleration. JSQKV jointly designs compression policy, numerical representation, and execution path. The key idea is simple: allocate different compression budgets to different tokens according to their importance to future attention, compress the remaining KV states with a token-aligned and numerically stable low-bit representation, and keep the KV cache compressed in memory as much as possible until dense tiles are required by the decode kernel. Through this joint design, JSQKV keeps the KV cache compact in memory while more effectively reducing memory traffic and improving end-to-end throughput.

The main contributions of this paper are as follows:

- We propose an attention-score-based differential sparsification policy that assigns different sparsity levels to different historical tokens at the token level, instead of applying a single global compression ratio.
- We design a dual-window online execution mechanism for autoregressive decoding, extending differential sparsification from the prefill stage to online decoding.
- We propose a Hadamard-based Per-Token-Tile quantization method to mitigate low-bit quantization instability caused by Key outliers and to better align quantization granularity with token-level compression decisions.
- We design a bitmap-based sparse-quant format and a matching decode kernel that keep the compressed KV cache compact in memory and turn compression into reduced memory traffic and end-to-end decoding speedup.

Experimental results show that JSQKV achieves a favorable balance between model quality and system efficiency under high compression ratios. On Meta-Llama-3-8B, with 70% KV cache sparsity and 2-bit quantization, JSQKV im-

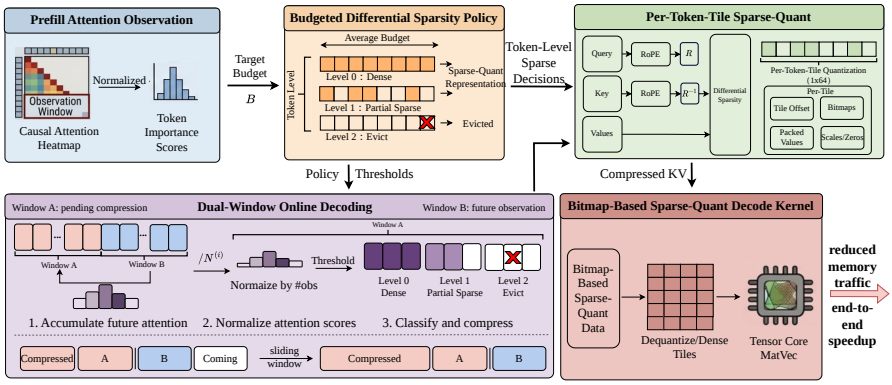


Fig. 1. Overview of JSQKV. The method first estimates token importance from prefill attention and instantiates a budgeted differential sparsity policy. It then extends this policy to autoregressive decoding through a dual-window online execution mechanism, represents retained KV states with Hadamard-based Per-Token-Tile quantization, and finally executes the compressed cache with a bitmap-based sparse-quant decode kernel to reduce memory traffic.

proves end-to-end throughput by approximately 44.2% over the dense baseline in a long-context decoding setting with batch size 4. These results show that efficient KV-cache compression requires joint optimization across compression policy, numerical representation, and execution path.

2 Method

2.1 Overview

Figure 1 summarizes the overall design of JSQKV. Rather than applying sparsification and quantization as two independent post-processing steps, JSQKV organizes them into a unified decode-stage pipeline in which the compression policy, low-bit representation, and execution path are designed together.

The method consists of four tightly coupled components. First, JSQKV uses prefill attention to estimate token importance and instantiates a budgeted differential sparsity policy, so that different historical tokens receive different retention budgets under a fixed average compression target. Second, because future attention is unavailable when a token is first generated during decoding, JSQKV introduces a dual-window online execution mechanism that delays compression until enough future evidence has been accumulated. Third, after token-level compression decisions are made, the retained KV states are stored in a token-aligned low-bit form using Hadamard-based Per-Token-Tile quantization, which improves robustness to key outliers while matching the downstream sparse-quant execution granularity. Finally, JSQKV uses a bitmap-based sparse-quant format

and a matching decode kernel to keep the KV cache compact in memory and turn compression into reduced memory traffic and practical decode-time speedup.

The remaining subsections describe these components in order. Section 2.2 presents the budgeted differential sparsity policy, Section 2.3 introduces the dual-window online execution mechanism, Section 2.4 describes the token-aligned quantization design, and Section 2.5 presents the sparse-quant storage format and decode kernel.

2.2 Budgeted Differential Sparsity Policy

The first challenge in KV-cache compression is to allocate a fixed compression budget across historical tokens with different future importance. A uniform sparsity ratio treats all tokens equally, even though some tokens carry persistent evidence for future decoding while others are rarely used again. Under the same average budget, such a uniform policy can waste budget on unimportant tokens and over-compress important ones. To address this issue, JSQKV estimates token importance from prefill attention and instantiates a three-level sparsity policy under an explicit average-budget constraint.

We use the attention distribution in prefill to estimate the relative importance of historical tokens. Let $\mathbf{A} \in \mathbb{R}^{T_q \times T_k}$ denote the causal attention matrix of the prompt. Following the observation-window idea in SnapKV [9], we use the last L_{obs} prompt tokens as an observation window. For a prefix token $j \le T_k - L_{\text{obs}}$, its importance score is defined as

$$s_j = \frac{T_k}{L_{\text{obs}}} \sum_{i=T_k-L_{\text{obs}}+1}^{T_k} A_{i,j}. \tag{1}$$

This score measures how much token j is attended by the queries in the observation window. Tokens inside the observation window are treated as highly important and are preserved by construction. The normalization factor T_k/L_{obs} makes the scores more comparable across prompts of different lengths.

Given the importance scores $\{s_j\}$, JSQKV assigns each token to one of three compression levels. Let $[p_0, p_1, p_2]$ denote the target proportions of Level 0, Level 1, and Level 2 tokens, where

$$p_0 + p_1 + p_2 = 1. \tag{2}$$

We compute two percentile thresholds,

$$\tau_{\text{high}} = \text{Percentile}(s, 1 - p_0), \tag{3}$$

$$\tau_{\text{low}} = \text{Percentile}(s, p_2), \tag{4}$$

and assign

$$\text{level}(j) = \begin{cases} 0, & s_j \ge \tau_{\text{high}}, \\ 1, & \tau_{\text{low}} \le s_j < \tau_{\text{high}}, \\ 2, & s_j < \tau_{\text{low}}. \end{cases} \tag{5}$$

The three levels correspond to different compression strengths. Level 0 tokens are kept dense. Level 2 tokens are fully removed from the cache. Level 1 tokens use feature-level sparsity and keep only the largest-magnitude features in their key and value vectors. For a key vector $\mathbf{k}_j \in \mathbb{R}^d$, we apply

$$\tilde{k}_{j,i} = \begin{cases} k_{j,i}, & i \in \mathrm{TopK}(|\mathbf{k}_j|, \lfloor (1-\rho_1)d \rfloor), \\ 0, & \text{otherwise}, \end{cases} \tag{6}$$

where ρ_1 is the feature-level sparsity ratio for Level 1 tokens. The same rule is applied to $\mathbf{v}_j$. In this way, important tokens retain more information, while less important tokens absorb more of the compression budget.

A fixed average sparsity budget does not uniquely determine the three-level policy. Different dense/partial/evict allocations can share the same average compression ratio but lead to different accuracy outcomes. We therefore instantiate the policy under an explicit budget constraint. Let

$$c = ([p_0, p_1, p_2], \rho_1) \tag{7}$$

denote one candidate configuration. With $\rho_0 = 0$ for Level 0 and $\rho_2 = 1$ for Level 2, the target average sparsity budget B satisfies

$$B = p_0\rho_0 + p_1\rho_1 + p_2\rho_2 = p_1\rho_1 + p_2. \tag{8}$$

For a given target budget B, we search over a small feasible set of (p_0, ρ_1) pairs and analytically recover the remaining proportions:

$$p_1 = \frac{1 - p_0 - B}{1 - \rho_1}, \qquad p_2 = 1 - p_0 - p_1. \tag{9}$$

Candidates with invalid proportions are discarded. We then select the final configuration on a small calibration split:

$$c^\star = \arg\max_{c \in \mathcal{C}(B)} \mathrm{Metric}(D_{\mathrm{cal}}; c), \tag{10}$$

where $\mathcal{C}(B)$ is the feasible candidate set under budget B. This procedure keeps the overall compression target fixed while allowing JSQKV to allocate the budget more carefully across dense, partially sparse, and evicted tokens.

2.3 Dual-Window Online Execution Mechanism

The policy in Section 2.2 relies on future attention observations that are available in prefill but unavailable when a token is first generated during decoding. As a result, the token-level compression policy cannot be applied immediately after a token is written into the KV cache. To make the same policy work during autoregressive decoding, JSQKV introduces a dual-window online execution mechanism that delays compression until each token has accumulated enough future attention evidence.

During decoding, the KV cache is organized into two consecutive windows of size W, as illustrated in Fig. 2. Window A stores tokens waiting to be compressed, while Window B stores newly generated tokens whose queries provide additional attention observations for Window A. This design avoids compressing a token too early, before it has accumulated enough future attention observations.

When a new query $\mathbf{q}_t$ is produced at decoding step t, we compute its attention to the tokens in Window A:

$$\boldsymbol{\alpha}_t = \mathrm{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_A^\top}{\sqrt{d}}\right), \tag{11}$$

where $\mathbf{K}_A$ denotes the key states of Window A. The resulting attention scores are accumulated in an attention buffer

$$\mathbf{Acc} \in \mathbb{R}^{N_b \times H \times W}, \tag{12}$$

where N_b is the batch size, H is the number of attention heads, and W is the window size.

When Window B becomes full, tokens in Window A have received different numbers of future observations, since earlier tokens are visible to more later queries. We therefore normalize the accumulated attention before reusing it as the token-importance signal. For the token at position $i \in \{0, \ldots, W-1\}$ in Window A, where i is zero-indexed, the number of observations is

$$N^{(i)} = 2W - 1 - i, \tag{13}$$

which yields the normalized score

$$\bar{\alpha}^{(i)} = \frac{\mathbf{Acc}[:, :, i]}{N^{(i)}}. \tag{14}$$

These normalized scores are then compared against the same thresholds $(\tau_{\text{high}}, \tau_{\text{low}})$ defined in Section 2.2, and each token in Window A is assigned to Level 0, Level 1, or Level 2 accordingly.

After the classification, JSQKV applies the corresponding compression rule to Window A: Level 0 tokens remain dense, Level 1 tokens are partially pruned, and Level 2 tokens are removed. The compressed Window A is then merged into the historical compressed region, the previous Window B becomes the new Window A, the accumulator is reset, and decoding continues with an empty new Window B. In this way, each token is observed before compression while the number of uncompressed tokens remains bounded by $O(W)$.

2.4 Per-Token-Tile Quantization with Hadamard Rotation

The remaining nonzero KV states still require a low-bit representation to reduce storage and memory traffic. Since token-level compression decisions are carried through the downstream sparse-quant operators, which are likewise organized as tiles along the token dimension, quantization should follow the same granularity.

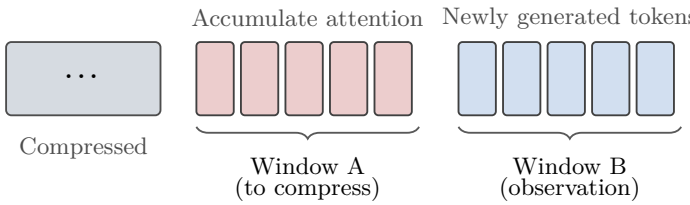


Fig. 2. Dual-window online execution mechanism during decoding. Window A accumulates attention statistics before compression, while Window B collects newly generated tokens and provides future observations.

We therefore adopt *Per-Token-Tile* quantization, which aligns the quantization unit with both token-level compression decisions and the tile-based decode path, while providing a fine-grained low-bit representation.

For a token vector $\mathbf{x}_t \in \mathbb{R}^d$ (either key or value), the feature dimension is partitioned into $M = d/g$ contiguous tiles of size g:

$$\mathbf{x}_t = [\mathbf{x}_{t,1}, \mathbf{x}_{t,2}, \ldots, \mathbf{x}_{t,M}], \qquad \mathbf{x}_{t,m} \in \mathbb{R}^g. \tag{15}$$

Each tile is quantized independently with its own scale $s_{t,m}$ and zero point $z_{t,m}$. The quantized tile is computed as

$$\mathbf{q}_{t,m} = \mathrm{clip}\left(\mathrm{round}\left(\frac{\mathbf{x}_{t,m} - z_{t,m}}{s_{t,m}}\right), 0,\, 2^b - 1\right), \tag{16}$$

where b is the quantization bitwidth. This design keeps the quantization parameters aligned with the token-level compression decision while reducing the local dynamic range within each tile.

A remaining challenge is that key states often contain outlier values, which can dominate the quantization range and make aggressive low-bit quantization unstable. To mitigate this effect, JSQKV applies a Hadamard rotation to queries and keys before quantization. Let $\mathbf{H} \in \mathbb{R}^{d\times d}$ denote the normalized Hadamard matrix. We rotate

$$\tilde{\mathbf{q}}_i = \frac{1}{\sqrt{d}}\mathbf{H}\mathbf{q}_i, \qquad \tilde{\mathbf{k}}_j = \frac{1}{\sqrt{d}}\mathbf{H}\mathbf{k}_j. \tag{17}$$

Because the Hadamard matrix is orthogonal, the attention score is preserved:

$$\tilde{\mathbf{q}}_i^\top \tilde{\mathbf{k}}_j = \mathbf{q}_i^\top \mathbf{k}_j. \tag{18}$$

The rotation therefore improves the distribution seen by the quantizer without changing the attention computation itself.

Figure 3 illustrates the effect of the Hadamard rotation on key states. After rotation, the heavy tail is noticeably suppressed, which reduces the dominance of outliers in low-bit quantization. At the same time, the rotated distribution does not collapse into a fully uniform one, and still preserves sufficient magnitude variation for subsequent magnitude-based feature sparsification.

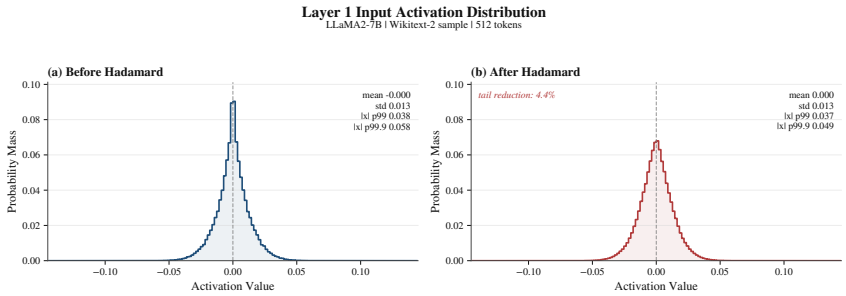


Fig. 3. Key-state activation distribution before and after Hadamard rotation on a LLaMA2-7B sample. The rotation suppresses heavy tails while preserving enough variation for feature-level sparsification.

Table 1. PPL of RTN and Hadamard rotation (*Rotate*) on WikiText-2 under quantization-only and sparse-quant settings. Hadamard rotation improves low-bit quantization stability, while remaining compatible with subsequent KV sparsification.

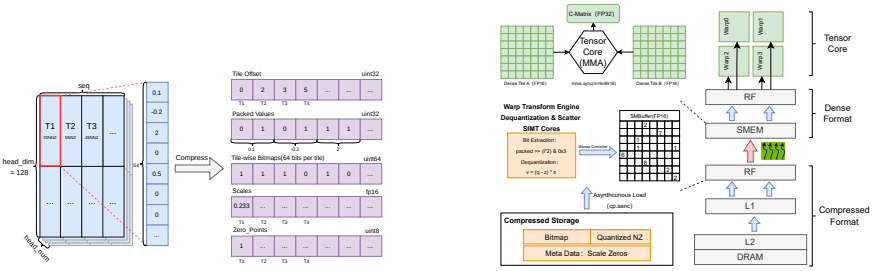
	4-bit		3-bit		2-bit	
Setting	RTN	Rotate	RTN	Rotate	RTN	Rotate
Quant only	5.22	**5.14**	6.57	**5.28**	115.55	**5.35**
+50% KV sparsity	**5.27**	5.69	6.57	**6.35**	115.55	**6.65**

Table 1 further supports this observation quantitatively on WikiText-2. Under quantization-only settings, Hadamard rotation consistently improves PPL at 4-bit, 3-bit, and especially 2-bit, confirming its effectiveness in stabilizing low-bit quantization. When 50% KV sparsity is further applied, the same trend remains clear at 3-bit and 2-bit, where Rotate still outperforms RTN by a visible margin. At 4-bit with added sparsity, however, the gain becomes marginal and slightly reverses, suggesting that when quantization error is already relatively small, the rotation-induced mixing may mildly weaken the magnitude separability used by subsequent sparsification. Overall, Per-Token-Tile quantization provides a fine-grained low-bit representation, while Hadamard rotation improves robustness to key outliers without largely disrupting subsequent sparsification.

2.5 Sparse-Quant Format and Decode Kernel

Compression quality alone is not enough if the resulting representation cannot be executed efficiently at decode time. We therefore co-design a sparse-quant format and its matching decode kernel, so that the compressed KV cache not only reduces storage cost, but also translates into practical decode acceleration.

Figure 4 gives an overview of this design. The left panel shows the sparse-quant format. Each 1×64 tile stores a bitmap for nonzero positions, packed low-bit values, a per-tile offset, and quantization metadata such as scales and



(a) bitmap-based sparse-quant format (b) Sparse-quant decode kernel

Fig. 4. Overview of the sparse-quant format and decode kernel. The left panel shows the bitmap-based sparse-quant format. The right panel shows the decode kernel that loads compressed tiles, performs unpacking and dequantization, reconstructs dense shared-memory tiles, and invokes Tensor Core matrix-vector computation.

zero points. This layout is designed to support high sparsity and low-bit storage while preserving a regular memory organization, thereby avoiding the irregular accesses and indexing overheads of conventional sparse formats.

The right panel shows the matching decode kernel. During decode, the kernel follows a *load-as-compressed, compute-as-dense* strategy. It first loads bitmaps and packed low-bit values from memory, then reconstructs the nonzeros through unpacking and dequantization, and scatters them into dense shared-memory tiles. Tensor Core matrix-vector computation is then performed on these reconstructed dense tiles. In this way, sparsity is handled only along the data-loading path, while the actual compute stage still uses regular dense operators.

This design is particularly important for decode-stage attention, which is strongly memory-bound. By keeping the KV cache compressed in memory, the format reduces memory traffic; by reconstructing dense tiles only along the compute path, the kernel avoids the control-flow divergence and irregular accesses that often limit sparse execution. As a result, the sparse-quant format and decode kernel together convert compressed KV representations into practical decode-time gains, rather than merely reducing storage footprint.

3 Experiments

3.1 Experimental Setup

To evaluate JSQKV, we conduct all experiments on the same hardware platform unless otherwise noted. The system uses an NVIDIA A100 80GB PCIe GPU, an Intel Xeon CPU, and 252GB host memory. Our implementation is based on Python 3.10, PyTorch 2.6.0, CUDA 12.4, HuggingFace Transformers, and FlashAttention [5]. For the critical compression and decoding path, JSQKV uses custom CUDA/Triton kernels for sparse-quant packing and decode-time reconstruction.

We use Meta-Llama-3-8B [6] as the main model and further evaluate on Llama2-7B, Mistral-7B, and Qwen2.5-7B for cross-model validation. Accuracy is measured on LongBench [2] with the official evaluation pipeline. Inputs are truncated to 8192 tokens; when an example is longer, we keep the first 4096 tokens and the last 4096 tokens. No extra training or fine-tuning is applied.

Budget instantiation protocol. For all differential-sparsity and joint sparse-quant settings, we do not fix a single universal three-level policy in advance. Instead, under each target average sparsity budget, we instantiate the policy using the calibration procedure described in Section 2.2. Concretely, for a target budget $B \in \{0.5, 0.7\}$, we construct a feasible candidate set of three-level configurations $c = ([p_0, p_1, p_2], \rho_1)$ satisfying the budget constraint in Equation (8), evaluate these candidates on a small held-out calibration split, and select the best-performing configuration under the same budget. We then re-evaluate the selected configuration on a disjoint validation split before running the full LongBench evaluation.

Unless otherwise noted, the calibration protocol keeps the remaining selection hyperparameters fixed and only searches the budget allocation itself, namely the dense/partial/evict proportions and the intermediate sparsity level. This protocol ensures that matched-budget comparisons are fair in two senses: the compared methods have the same target average sparsity, and the differential policy is instantiated through the same calibration procedure rather than through hidden task-specific manual tuning. For task-adaptive results, calibration and validation are performed separately for each task; for shared-policy results, one calibrated configuration is selected and then reused across the evaluation tasks.

In the experiments reported below, we use a small calibration subset and a disjoint validation subset drawn from the same benchmark pool with fixed random seeds. This lightweight protocol is sufficient to expose task-level differences in preferred budget allocation while keeping the tuning cost negligible compared with full evaluation.

We answer three questions. First, does differential sparsity outperform a uniform sparsity budget under the same average compression target? Second, does the full joint design outperform a sequential sparsity-plus-quantization baseline? Third, do compression gains translate into end-to-end decoding speedups? To answer them, we compare dense decoding, MUSTAFAR-style uniform sparsity [8], and the differential sparsity policy of JSQKV under 50% and 70% average sparsity budgets. For the full method, we compare JSQKV against a sequential baseline that applies uniform sparsity and then KIVI-style low-bit quantization [10]; we denote this baseline by **M+K**. We report 50/50 and 70/70 target sparsity ratios for K and V caches, each combined with 4-bit and 2-bit quantization. For efficiency, we measure end-to-end throughput and compression overhead on Meta-Llama-3-8B with 4096 input tokens, 256 output tokens, and batch sizes from 1 to 8.

Table 2. Differential sparsity versus uniform sparsity on selected LongBench tasks for Meta-Llama-3-8B.

Budget	Method	NarrativeQA	Qasper	GovReport	TREC	LCC	Avg.
50%	Dense	23.45	42.18	29.91	74.00	57.12	45.33
	MUSTAFAR	23.28	41.21	28.36	73.02	56.28	44.43
	Differential sparsity	**23.36**	**41.56**	**28.79**	**73.40**	**56.72**	**44.77**
70%	Dense	23.45	42.18	29.91	74.00	57.12	45.33
	MUSTAFAR	22.97	40.89	24.38	70.86	54.21	42.66
	Differential sparsity	**23.18**	**41.02**	**25.10**	**71.58**	**55.02**	**43.21**

Table 3. Detailed LongBench results on Meta-Llama-3-8B. "50/50" and "70/70" denote target sparsity ratios for K and V caches.

Setting	NarrativeQA	HotpotQA	GovReport	TREC	PCount	LCC	Avg.
Dense	23.45	46.06	29.91	74.00	4.50	57.12	39.17
M+K, 50/50 + 4-bit	23.18	45.68	28.94	73.41	4.28	56.21	38.62
JSQKV, 50/50 + 4-bit	**23.28**	**45.81**	**29.08**	**73.55**	**4.36**	**56.35**	**38.74**
M+K, 70/70 + 4-bit	23.02	45.31	24.86	71.08	4.01	54.52	37.13
JSQKV, 70/70 + 4-bit	**23.14**	**45.45**	**25.07**	**71.24**	**4.12**	**54.71**	**37.29**
M+K, 50/50 + 2-bit	22.54	45.14	28.61	73.32	4.18	48.74	37.09
JSQKV, 50/50 + 2-bit	**22.83**	**45.33**	**29.05**	**73.58**	**4.29**	**49.10**	**37.36**
M+K, 70/70 + 2-bit	22.48	44.72	23.84	70.65	3.96	45.70	35.23
JSQKV, 70/70 + 2-bit	**22.71**	**44.88**	**24.18**	**71.02**	**4.10**	**46.24**	**35.52**

3.2 Accuracy Results

Differential sparsity alone. We first evaluate the sparsity policy without quantization or Hadamard rotation. The goal is to test whether attention-guided budget allocation is better than a uniform sparsity ratio under the same average sparsity target. Table 2 reports results on a representative LongBench subset. The results show that differential sparsity consistently improves over MUSTAFAR, and that the gain becomes larger at 70% sparsity. This supports the main intuition of JSQKV: compression budget should be concentrated on tokens that are more likely to remain useful during future decoding.

Joint sparsity-quantization results. Table 3 reports detailed LongBench results on Meta-Llama-3-8B for the full joint framework. The results show that JSQKV outperforms the sequential M+K baseline across all four compression settings. The gains are modest under milder compression, but become more visible when the budget becomes tighter, especially on GovReport and LCC. This trend is consistent with the motivation of JSQKV: as the compression ratio increases, the benefit of aligning sparsity, quantization, and runtime execution also increases.

To test whether the trend is specific to one architecture, Table 4 summarizes the average LongBench score on four additional models. The results show that the advantage of JSQKV over M+K remains consistent across Llama-2, Mistral, and Qwen variants. This suggests that the benefit comes from the joint design itself rather than from a model-specific artifact.

Table 4. Cross-model LongBench averages. "M+K" denotes the sequential MUSTAFAR+KIVI baseline.

Model	Dense	M+K 50/50 4-bit	JSQKV 50/50 4-bit	M+K 70/70 4-bit	JSQKV 70/70 4-bit	M+K 50/50 2-bit	JSQKV 50/50 2-bit	M+K 70/70 2-bit	JSQKV 70/70 2-bit
Llama2-7B	27.49	27.02	**27.19**	26.18	**26.37**	25.98	**26.29**	24.73	**25.08**
Llama2-13B	27.66	27.20	**27.40**	26.42	**26.71**	26.09	**26.40**	24.98	**25.31**
Mistral-7B	35.70	35.08	**35.31**	34.02	**34.37**	33.56	**33.94**	32.18	**32.61**
Qwen2.5-7B	36.88	36.24	**36.49**	35.11	**35.52**	34.86	**35.28**	33.31	**33.77**

Table 5. End-to-end throughput (tokens/s) on Meta-Llama-3-8B with 4096 input tokens and 256 output tokens.

Batch Size	Dense	Sparse50	Sparse70	Sparse50+2bit	Sparse70+2bit
1	28.55	17.16	16.84	23.24	23.38
2	39.81	31.75	30.56	42.76	42.37
3	49.15	45.15	46.18	55.45	57.81
4	55.83	53.80	57.63	64.70	66.42
5	56.67	66.93	65.96	72.02	74.22
6	56.81	77.17	78.01	76.80	80.38
7	59.69	86.52	83.53	80.47	84.06
8	62.80	94.51	96.43	86.98	87.12

3.3 Efficiency Results

End-to-end throughput. Table 5 and Fig. 5 report end-to-end decoding throughput on Meta-Llama-3-8B. Dense decoding remains strongest at batch size 1, where compression overhead is not yet amortized. Starting from batch size 2, the quantized path becomes competitive and often superior. The strongest region for the 70/70 + 2-bit configuration is the small-to-medium batch regime: at batch size 6, JSQKV reaches 80.38 tokens/s, compared with 56.81 tokens/s for dense decoding and 78.01 tokens/s for sparse-only decoding. At larger batch sizes, the workload becomes more compute-heavy and the benefit of reduced KV traffic narrows. Overall, the throughput results show that joint compression can translate into practical speedups once the workload becomes sufficiently bandwidth-sensitive.

Compression overhead. Compression is useful only if its own overhead remains controlled. Table 6 reports representative prefill-time compression statistics. The results show that the 2-bit sparse-quant path achieves substantially higher compression ratios than sparse-only baselines, while also reducing write-back cost enough to keep compression time competitive. For example, at batch size 8, Sparse70 reduces the KV cache to 1964.29MB, whereas Sparse70+2bit further reduces it to 1176.25MB and also lowers compression time from 5411.29ms to 4293.18ms. This indicates that the efficiency benefit of JSQKV comes not only from lower decode-time bandwidth demand, but also from a more compact compression pipeline.

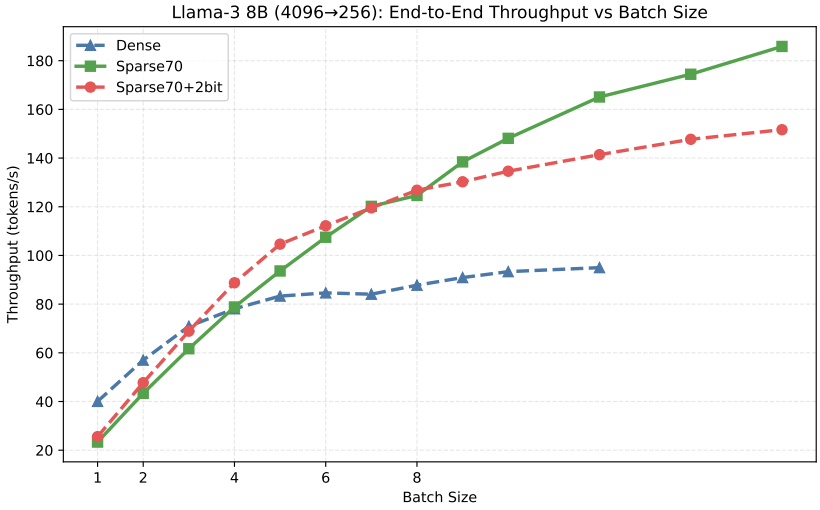


Fig. 5. Representative end-to-end throughput on Meta-Llama-3-8B with 4096 input tokens and 256 output tokens. The compressed 70/70 + 2-bit path outperforms dense decoding in the small-to-medium batch regime.

3.4 Component Ablation

To understand which parts of the framework matter most, Table 7 reports an ablation under the most aggressive tested configuration, namely 70/70 sparsity with 2-bit quantization. We compare the full method against three degraded variants: replacing differential sparsity with a uniform sparsity budget, removing Hadamard rotation, and replacing per-token-tile quantization with a coarser alternative. The results show that all three changes hurt performance. The largest drop comes from removing Hadamard rotation, which confirms that low-bit key quantization is highly sensitive to outliers. The gap between the full method and the uniform-sparsity variant also confirms that better budget allocation remains important even after quantization is introduced.

Taken together, these results support the main thesis of JSQKV: high-ratio KV-cache compression works best when token selection, numeric representation, and execution path are designed as one system.

4 Discussion and Future Work

Although JSQKV achieves a favorable accuracy-efficiency trade-off in long-context scenarios, several limitations remain for future work.

Firstly, its effectiveness relies on explicit token-importance evaluation rather than a uniform sparsity budget, yet the current design still mainly operates at the token level and does not capture finer-grained sensitivity across layers, attention heads, or Key/Value representations. A more fine-grained joint design

Table 6. Representative compression overhead and KV-cache size for Meta-Llama-3-8B (input length 4096). Compression ratio is original KV size divided by compressed KV size.

Batch Size	Method	KV-Cache Size (MB)	Compression Ratio	Compression Time (ms)
1	Sparse50	2720.16	1.51×	740.80
	Sparse70	1964.29	2.09×	716.30
	Sparse50+2bit	1261.16	3.25×	545.18
	Sparse70+2bit	1176.25	3.48×	552.86
8	Sparse50	2720.16	1.51×	5688.23
	Sparse70	1964.29	2.09×	5411.29
	Sparse50+2bit	1261.16	3.25×	4317.88
	Sparse70+2bit	1176.25	3.48×	4293.18

Table 7. Component ablation on Meta-Llama-3-8B under 70/70 sparsity and 2-bit quantization.

Method	NarrativeQA	HotpotQA	GovReport	LCC	Avg.
Full JSQKV	**22.71**	**44.88**	**24.18**	**46.24**	**34.50**
w/o Differential Sparsity	22.48	44.72	23.84	45.70	34.19
w/o Hadamard	22.21	44.16	22.97	44.61	33.49
w/o Per-Token-Tile	22.33	44.29	23.18	44.98	33.70

of importance modeling and quantization is therefore worth exploring. Secondly, the joint sparse-quant path is not advantageous under all serving conditions. Its largest throughput gains appear in small- to medium-batch scenarios, while at larger batch sizes the benefit of compression is increasingly offset by dequantization, metadata handling, and higher compute pressure. Designing more efficient sparse-quant operators thus remains an important direction. Finally, the current evaluation scope is still limited. Although experiments cover multiple model families and representative long-context settings, further study is needed to verify whether the proposed method remains effective in ultra-long-context scenarios, on heterogeneous accelerators, and in production-like deployment environments.

5 Conclusion

JSQKV reduces KV-cache cost through differential sparsification, stabilized low-bit quantization, and a bitmap-based sparse-quant decode kernel. Evaluation results show consistent gains over a sequential sparsity-plus-quantization baseline under matched compression budgets, and they further show that compression savings can translate into practical throughput gains during decoding. Overall, the results support treating KV-cache compression as a systems-and-algorithms co-design problem rather than as a simple composition of independent post-processing steps.

References

1. Ashkboos, S., Mohtashami, A., Croci, M.L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., Hensman, J.: Quarot: Outlier-free 4-bit inference in rotated llms. arXiv preprint arXiv:2404.00456 (2024)
2. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508 (2023), https://arxiv.org/abs/2308.14508
3. Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., Xiao, W.: Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. arXiv preprint arXiv:2406.02069 (2024), https://arxiv.org/abs/2406.02069
4. Chen, Z., Yuan, K., Wei, H., Li, L., Shen, W., Su, Z., Yu, H.: Rotatekv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations (2025), https://arxiv.org/abs/2501.16383
5. Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Re, C.: Flashattention: Fast and memory-efficient exact attention with io-awareness. Advances in Neural Information Processing Systems **35** (2022)
6. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
7. Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M.W., Shao, Y.S., Keutzer, K., Gholami, A.: Kvquant: Towards 10 million context length llm inference with kv cache quantization. arXiv preprint arXiv:2401.18079 (2024)
8. Joo, D., Hosseini, H., Hadidi, R., Asgari, B.: Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference. arXiv preprint arXiv:2505.22913 (2025), https://arxiv.org/abs/2505.22913
9. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., Chen, D.: Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469 (2024), https://arxiv.org/abs/2404.14469, version 2, June 17, 2024
10. Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., Hu, X.: Kivi: A tuning-free asymmetric 2bit quantization for kv cache. arXiv preprint arXiv:2402.02750 (2024)
11. Lv, B., Zhou, Q., Ding, X., Wang, Y., Ma, Z.: Kvpruner: Structural pruning for faster and memory-efficient large language models. arXiv preprint arXiv:2409.11057 (2024), https://arxiv.org/abs/2409.11057
12. Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M.: Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453 (2024), https://arxiv.org/abs/2309.17453
13. Xu, Y., Jie, Z., Dong, H., Wang, L., Lu, X., Zhou, A., Saha, A., Xiong, C., Sahoo, D.: Think: Thinner key cache by query-driven pruning. In: International Conference on Learning Representations (2025), https://arxiv.org/abs/2407.21018, spotlight
14. Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., Chen, B.: H_2o: Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048 (2023), https://arxiv.org/abs/2306.14048